BÁO CÁO KỸ THUẬT LAB 3

Xây dựng, Kiểm thử và Kiểm chứng Độc lập Pipeline Dữ liệu Lakehouse

---

Nhóm thực hiện: `bigdata-cloud-lab3` | **Môi trường:** EC2 (Spark / Delta Lake)
Repository: `https://github.com/thanhdat24020062/bigdata-cloud-lab3`

1. Tổng quan và Phân công nhiệm vụ

Trong Lab 3, nhóm triển khai quy trình xử lý dữ liệu theo kiến trúc Lakehouse (Bronze – Silver – Gold) kết hợp cơ chế kiểm soát tính toàn vẹn đầu vào, cập nhật tăng dần (Incremental Merge), đánh giá hiệu năng lưu trữ và ràng buộc phát hành (Release Binding). Để tránh tình trạng tự kiểm tra chéo thiếu khách quan, công việc được chia thành 5 vai trò độc lập tương ứng với các chặng của pipeline:

Vai trò	Thành viên	Phụ trách chính	Đầu ra & Bằng chứng tiêu biểu
Role A	Thành Đạt	Input Acceptance & Runtime Configuration	Xác minh 16/16 file đầu vào, kiểm tra handoff, preflight EC2, tamper test (`evidence/role_a/`).
Role B	Xuân Tiến	Lakehouse Semantics & Gold Rollup	Hàm `build_gold`, bootstrap snapshot v0, chứng minh bảo toàn B03/B04 (`gold_baseline.json`).
Role C	Dương	Incremental Merge & Replay	Hàm `choose_revisions`, `merge_silver`, xử lý batch-02, kiểm thử replay & invalid batch.
Role D	Cảnh	Analytics & Storage Layout Benchmark	Truy vấn Gold v1 (Q1–Q3), trực quan hóa, benchmark bố cục A/B (`role_D/benchmark_run.log`).
Role E	Xuân Đào	Release Governance & Lineage	Hàm `validate_release_binding`, 5 cổng kiểm duyệt R01–R05, đóng gói `release_candidate/`.

2. Chi tiết thực hiện theo từng vai trò

2.1. Role A (Thành Đạt): Tiếp nhận dữ liệu đầu vào và Kiểm tra môi trường

Role A đảm nhiệm chốt chặn đầu tiên (Input Gate) nhằm bảo đảm pipeline vận hành trên đúng nguồn dữ liệu, toàn vẹn ở mức byte và hạ tầng thực thi sẵn sàng trước khi chạy các tác vụ Spark/Delta:

- **Xác minh tính toàn vẹn dữ liệu (Input Integrity):** Thiết lập cơ chế đối chiếu mã băm SHA-256 cho toàn bộ 16 đối tượng đầu vào trên máy chủ EC2 (`/home/ubuntu/repo-lab3`) dựa trên bộ manifest chuẩn (`trusted_lab3_inputs.json` và `manifests/lab3_inputs_convenience.json`). Kết quả đạt 16/16 `PASS`, loại bỏ rủi ro sai lệch dữ liệu ngay từ đầu.
- **Xác thực định danh Handoff từ Lab 2:** Kiểm tra tệp `data/handoff/handoff_manifest.json`, ghi nhận tập dữ liệu gồm 9.610 bản ghi, 10 cột từ nguồn `research-telemetry-v1` với mã băm logic `c50f8252...a0bf0d`. Nhóm cũng làm rõ nhãn `course_fallback_not_student_approval` để sử dụng đúng ngữ cảnh, tránh nhầm lẫn với bản release đã phê duyệt chính thức.
- **Kiểm tra điều kiện hạ tầng (Runtime Preflight):** Rà soát môi trường EC2, phiên bản Python, thư viện Spark/Delta Lake, khả năng khởi tạo worker và quyền đọc dữ liệu. Toàn bộ trạng thái được xuất ra các tệp `ec2_environment.txt`, `preflight.json`, `runtime_preflight.json` và `worker_probe.json` để tách biệt lỗi hạ tầng với lỗi logic xử lý.
- **Truy vết và thử nghiệm chống giả mạo:** Lưu nhật ký sự kiện tại `lineage.jsonl`, cố định phiên bản mã nguồn trong `code_versions/` và phối hợp với Role C thử nghiệm can thiệp trái phép trên

4 tập mẫu (`observations_a.csv`, `observations_b.jsonl`, `sensors.csv`, `qa_reference.csv`). Hệ thống từ chối chính xác toàn bộ bản sao bị sửa đổi với mã lỗi `INPUT_INTEGRITY`, đồng thời ghi nhận lưu ý bảo vệ trust anchor tại thư mục `/work`.

2.2. Role B (Xuân Tiến): Tổng hợp tầng Gold và Kiểm chứng bảo toàn

Role B phụ trách thiết kế tầng dữ liệu phân tích (Gold) và chứng minh tính đúng đắn về mặt toán học khi tổng hợp dữ liệu từ tầng Silver:

- **Triển khai logic tổng hợp Gold:** Lập trình hàm `build_gold` trong `templates/student_impl.py` để gom nhóm dữ liệu theo `site` và `event_date`. Các phép tính tổng hợp được ép kiểu `DecimalType` chính xác theo lược đồ quy định trước khi vật lý hóa bảng phân tích.
- **Kiểm thử khởi tạo (Bootstrap Verification):** Chạy tiến trình bootstrap để khởi tạo thành công snapshot ban đầu (version 0) cho cả ba tầng Bronze, Silver và Gold. Trạng thái và dữ liệu nền được lưu vết đầy đủ tại `evidence/state.json`, `evidence/silver_baseline.jsonl` và `evidence/gold_baseline.json`.
- **Giải trình phân tích toán học (B03 & B04):** Hoàn thiện phân tích trong `templates/experiment_report.md`, chứng minh định luật bảo toàn số dòng và giá trị tích lũy từ Silver lên Gold. Đồng thời làm rõ nguyên nhân chênh lệch thống kê giữa trung bình không trọng số (Unweighted Mean) và trung bình có trọng số theo số quan sát (Observation-Weighted Mean).

2.3. Role C (Dương): Cập nhật tăng dần, Replay và Chính sách từ chối Batch lỗi

Role C xây dựng cơ chế đồng bộ dữ liệu biến động theo thời gian thực vào bảng Silver và bảo đảm tính nhất quán khi chạy lại (idempotence):

- **Logic cập nhật tăng dần (Incremental Merge):** Hoàn thiện hai hàm `choose_revisions` và `merge_silver` trong `templates/student_impl.py`. Hàm `choose_revisions` chọn duy nhất một bản ghi thắng cho mỗi `record_id` dựa trên thứ tự ưu tiên: `ingest_time_utc` giảm dần, sau đó đến `source_object` và `source_row` tăng dần để bảo đảm kết quả tất định. Hàm `merge_silver` sử dụng Delta `MERGE` để chèn khóa mới và chỉ cập nhật khi dữ liệu đến mới hơn dữ liệu hiện có.
- **Thực thi và đối soát `batch-02`:** Chạy luồng incremental (`attempt-c02`) trên 140 dòng của tệp `data/changes/changes_batch02.csv`. Hệ thống lọc ra 130 bản ghi thắng (gồm 100 bản ghi chèn mới, 20 bản ghi cập nhật) và loại bỏ 10 bản ghi cũ (stale); 10 khóa trùng lặp được phân định chính xác qua `source_row`. Sau khi merge, Bronze đạt 9.750 dòng, Silver đạt 9.710 dòng (phiên bản 1) và Gold được dựng lại từ 16 lên 20 nhóm (`evidence/incremental.json`, `silver_after_batch02.jsonl`).
- **Kiểm chứng Replay và Time Travel:** Nạp lại cùng một batch để kiểm tra tính lũy đẳng. Kết quả cho thấy Bronze giữ nguyên 9.750 dòng; Silver tuy tăng thêm một commit rỗng (từ phiên bản 1 lên 2) nhưng vẫn giữ nguyên mã băm logic (`d0e76f8b...162a`). Truy vấn Time Travel về Silver phiên bản 0 trả về đúng 9.610 dòng handoff ban đầu (`silver_replay.jsonl`, `silver_time_travel.jsonl`).
- **Chính sách chặn Batch lỗi (Invalid-batch Policy):** Thiết lập cơ chế kiểm tra toàn bộ batch trước khi ghi (all-or-nothing). Cả 3 kịch bản kiểm thử đột biến gồm sai giá trị (`VALUE`), thiếu trường bắt buộc (`REQUIRED_VALUE`) và thêm cột ngoài danh mục (`SCHEMA_ALLOWLIST`) đều bị chặn hoàn toàn mà không làm thay đổi dữ liệu trong bảng (`evidence/input_contract_tests.json`).

2.4. Role D (Cảnh): Phân tích tầng Gold và Thí nghiệm bố cục lưu trữ

Role D khai thác dữ liệu từ tầng Gold sau cập nhật và đánh giá tác động của việc phân vùng dữ liệu (partitioning) đối với hiệu năng truy vấn:

- **Phân tích truy vấn tầng Gold:** Thực thi 3 truy vấn SQL trên Gold phiên bản 1 và lưu kết quả tại `evidence/Q1_site_day.json`, `Q2_weighted_day.json` và `Q3_above25_rate.json`. Ở truy vấn

Q2, tổng số quan sát ghi nhận 9.710 (khớp hoàn toàn với số dòng của Silver v1); trung bình có trọng số và trung bình cộng giữa các site chỉ lệch nhau ở ngày `2026-02-02` ($+0{,}001794$) và `2026-02-05` ($-0{,}000321$) do mật độ quan sát giữa các trạm không đồng đều.

- **Trực quan hóa dữ liệu:** Xây dựng kịch bản `role_D/replot.py` đọc trực tiếp từ các tệp JSON kết quả để xuất hai biểu đồ: `evidence/daily_mean.png` (trung bình ngày có trọng số kèm số lượng quan sát n) và `evidence/late_fraction.png` (tỉ lệ bản ghi đến muộn n_{late}/n theo từng trạm và ngày).
- **Thí nghiệm bố cục lưu trữ (Layout Benchmark):** So sánh hiệu năng truy vấn lọc theo điều kiện `event_date = 2026-02-06` trên tập dữ liệu tổng hợp 971.000 dòng (nhân bản 100 lần từ Silver v1) giữa bố cục A (không phân vùng, 32 tệp) và bố cục B (`partitionBy(event_date)`, 5 tệp). Đo đạc 6 lượt chạy xen kẽ (A-B, B-A, A-B) ghi nhận trung vị thời gian của A là $0{,}527s$ và B là $0{,}402s$ (tỉ lệ $\approx 1{,}31$). Phân tích kế hoạch thực thi cho thấy B tận dụng `PartitionFilters` trong khi A dùng `PushedFilters`; đồng thời nhóm cũng chỉ rõ giới hạn thực nghiệm (chạy cục bộ một máy, chênh lệch dung lượng bảng $16{,}4MB$ so với $0{,}53MB$) nên không quy toàn bộ mức cải thiện tốc độ cho cơ chế cắt tỉa phân vùng (`role_D/benchmark_run.log`).

2.5. Role E (Xuân Đào): Kiểm duyệt phát hành và Quản lý minh chứng

Role E chịu trách nhiệm kiểm soát chất lượng đầu ra cuối cùng, bảo đảm mọi bản phát hành đều tuân thủ quy tắc quản trị và truy vết đầy đủ:

- **Xây dựng hàm kiểm duyệt phát hành:** Lập trình hàm `validate_release_binding` trong `templates/student_i` trả về cấu trúc `(bool, list)`. Thay vì dừng ngay khi gặp lỗi đầu tiên (return early), hàm gom toàn bộ các lỗi vi phạm để cung cấp báo cáo kiểm duyệt đầy đủ trong một lần chạy.
- **Kiểm thử 5 cổng kiểm duyệt (Release Gate Tests):** Thử nghiệm pipeline qua 5 kịch bản phát hành từ R01 đến R05 (`evidence/release_gate_tests.json`). Hệ thống chặn đứng chính xác các lỗi: lẫn lộn phiên bản bảng (`mixed version`), người viết tự duyệt (`self-review`), thiếu người quản lý dữ liệu (`missing steward`) và sai mã băm mã nguồn (`wrong code`). Cả 5/5 kịch bản đều đạt trạng thái `PASS`.
- **Đóng gói bản phát hành (Release Candidate):** Xuất gói chứng nhận tại thư mục `evidence/release_candidate/` gồm `catalog.json`, `gold.json`, `release_manifest.json` và `snapshot_binding.json`. Gói phát hành ghi nhận rõ phân quyền tách bạch (Author: Thanh Dat, Reviewer: Xuan Dao, Steward: Xuan Tien) và khóa chặt với phiên bản dữ liệu (`Silver=1`, `Gold=1`) cùng mã băm SHA-256 của mã nguồn.
- **Hợp nhất nhật ký hệ thống (Lineage):** Ghi nhận sự kiện hoàn tất toàn vẹn của pipeline Spark (`eventType: COMPLETE`) vào tệp `evidence/lineage.jsonl`.

3. Tổng hợp kết quả Đối chiếu chéo (Independent Cross-check)

Mỗi thành viên đều thực hiện kiểm tra độc lập kết quả của một vai trò khác mà không phụ thuộc vào báo cáo tự xác nhận của người thực hiện:

4. Kết luận

Qua quá trình triển khai và kiểm chứng chéo giữa 5 thành viên, nhóm `bigdata-cloud-lab3` đã hoàn thiện đầy đủ các yêu cầu kỹ thuật của Lab 3. Dữ liệu đầu vào được kiểm soát chặt chẽ bằng mã băm SHA-256; các thao tác cập nhật tăng dần trên Delta Lake bảo đảm tính tất định, lũy đẳng và truy vết lịch sử thông qua Time Travel. Các kết quả phân tích tầng Gold, thực nghiệm bố cục lưu trữ và quy trình đóng gói bản phát hành đều có bằng chứng thực thi độc lập lưu trữ đầy đủ trong thư mục `evidence/` của repository.

Người kiểm tra	Đối tượng	Phương pháp và nội dung đối chiếu	Kết quả
Role A (Đạt)	Toàn nhóm (B, D, E)	Cung cấp và đối soát mã băm chuẩn (`trusted_lab3_inputs`, `handoff_manifest`) để bảo đảm các bước đều chạy trên cùng một snapshot.	Đã đồng bộ
Role B (Tiến)	Role C (Incremental)	Nghiệm thu độc lập `evidence/incremental.json`, xác nhận logic từ chối bản ghi cũ (stale) và thứ tự tie-breaking khi trùng khóa; lưu biên bản tại `manual_checks.csv`.	`PASS`
Role C (Dương)	Role D (Benchmark)	Tự viết mã Python thuần (độc lập với Spark và code của Role D) tính đáp án từ Silver v1; sau khi chia hệ số nhân bản 100, kết quả khớp 16/16 giá trị với Role D (`role_c_cross_check_layout_ab.md`).	`PASS (16/16)`
Role D (Cảnh)	Role B (Phân tích B03)	Dự đoán lý thuyết trước khi chạy kịch bản kiểm tra độc lập `role_D/check_D_local.py`; kết quả trong `check_D_output.txt` khớp hoàn toàn với `Q2_weighted_day.json`.	`PASS`
Role E (Đào)	Toàn bộ Release	Đối soát chéo phiên bản bảng, mã băm code và tách bạch vai trò Author/Reviewer/Steward để loại bỏ rủi ro trộn lẫn snapshot trước khi đóng gói nộp bài.	`PASS`